COS40007 Artificial Intelligence Engineering

Swinburne University of Technology | 2026 HE Semester (Sep)

Applied Project – Group Task 1

System Blueprint & Project Planning

AI-Driven Predictive Maintenance System for Milling Machines

Dataset: AI4I 2020 Predictive Maintenance Dataset (Matzka, 2020a)

Studio Session No.	5	
Group No. (given by tutor)	HCM-6	
Group Member	Role	Student ID
Phan Quang Huy	ML Engineer & Data Scientist	106219394
Đặng Võ Hoàng Long	MLOps Engineer & Data Engineer	104808745
Submission date	22 October 2026	
Group GitHub repository	https://github.com/long-blip/COS40007-HCM6	

Acknowledgement of Country

We respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging. We are honoured to recognise our connection to Wurundjeri Country, history, culture, and spirituality through these locations, and strive to ensure that we operate in a manner that respects and honours the Elders and Ancestors of these lands. We also respectfully acknowledge Swinburne's Aboriginal and Torres Strait Islander staff, students, alumni, partners and visitors. We also acknowledge and respect the Traditional Owners of lands across Australia, their Elders, Ancestors, cultures, and heritage, and recognise the continuing sovereign connections to their lands and waters.

Contents

1. Group Agreement and Project Plan
2. System Blueprint Diagram
3. MLOps & DevOps Integration (High-Level Design)
4. Project Management Setup (Agile Tools)
5. Contribution Statement
6. Group GitHub Repository
7. GenAI Declaration
8. References

1. Group Agreement and Project Plan

- **Studio Session No.:** 5
- **Group No. (given by tutor only):** HCM-6
- **Group members:** Phan Quang Huy (106219394); Đặng Võ Hoàng Long (104808745)

Project overview

Unplanned machine breakdowns cause costly downtime in manufacturing. Our project builds an AI system that predicts machine failures from sensor readings so that maintenance can be scheduled before a failure happens. We use the AI4I 2020 Predictive Maintenance Dataset: 10,000 records of a simulated milling process with six process and sensor features, a binary "Machine failure" label and five failure-mode labels (TWF, HDF, PWF, OSF, RNF).

- **Primary ML task:** binary classification – will the machine fail?
- **Secondary ML task:** failure-type prediction – which failure mode(s) are likely? Because failure types can overlap, this is treated as a multilabel problem.
- **Target users:** maintenance engineers and factory/operations managers.
- **Success criteria:** Recall ≥ 0.80 on the failure class, F1-score ≥ 0.70, a working dashboard, and an automated retraining pipeline on GitHub Actions.

Group style and roles

We are a two-person team, so each member owns one side of the system end to end and reviews the other's work. Huy owns the model side; Long owns the data and automation side. Both members write code, documentation and tests.

Member	Role	Main responsibilities
Phan Quang Huy	ML Engineer & Data Scientist	• Exploratory data analysis (EDA) and feature engineering • Model training, tuning and evaluation • Drift detection logic and model monitoring metrics • Streamlit dashboard (user interface)
Đặng Võ Hoàng Long	MLOps Engineer & Data Engineer	• Data ingestion, validation and preprocessing pipeline • GitHub repository management and code review rules • CI/CD with GitHub Actions, data/model versioning with DVC • Monitoring and retraining automation

Responsibility	Who
Scrum Master	Huy for Sprints 1 and 3; Long for Sprints 2 and 4. The Scrum Master runs meetings, keeps the Trello board up to date and tracks deadlines.
Responsible for submitted deliverables	Huy compiles and submits each deliverable; Long reviews it before submission. Both members check the final file together.
Asks questions on behalf of the group	The Scrum Master of the current sprint contacts the tutor (Canvas/Teams or in the studio class).

Timeline (Sprint plan)

The project is divided into four sprints. Each sprint ends on a Canvas assessment date, and we aim to finish every deliverable at least one day before its deadline.

Sprint	Dates	Goal	Key deliverables	Assessment
Sprint 1	19 Oct – 22 Oct	Planning & design	EDA insights, system blueprint, MLOps/DevOps design, Trello board, GitHub repository	Task 1 (22 Oct)
Sprint 2	23 Oct – 5 Nov	Data pipeline & baseline model	Preprocessing and feature engineering scripts, baseline model (model.py), first GitHub Actions workflow (train.yml) producing metrics	Task 2 (5 Nov)
Sprint 3	6 Nov – 26 Nov	Automation, monitoring & UI	Model tuning, DVC with remote storage, automated retraining (data push, schedule, manual), drift detection, Streamlit dashboard	Task 3 (26 Nov)
Sprint 4	27 Nov – 4 Dec	Finalisation & delivery	Final testing, final report, presentation slides, live demo rehearsal	Final Report (3 Dec), Presentation & Demo (4 Dec)

Key decision points

By	Decision / checkpoint
19 Oct	Project topic, roles, sprint plan and pipeline components agreed (Sprint 1 planning meeting).
20 Oct	EDA insights shared; GitHub repository and Trello board created.
21 Oct	Blueprint diagram and MLOps/DevOps design drafted; user stories written.
22 Oct	Task 1 report reviewed by both members and submitted before 20:00.
30 Oct	Baseline model chosen (Random Forest vs XGBoost) based on validation Recall/F1.
4 Nov	GitHub Actions training workflow working on main; Task 2 submitted.
20 Nov	Retraining triggers, DVC remote and drift checks working end to end.
1 Dec	Feature freeze: only bug fixes, report and demo preparation after this date.

Communication Plan

Channel / tool	Used for	Frequency
Zalo group chat	Quick questions, daily progress updates, blockers	Daily (short update every evening)
Microsoft Teams meeting	Sprint planning, progress check and sprint review	Once a week (Friday, 20:00, ~30–45 min) + a short check-in after each studio class
Trello board	User stories, backlog, task status and ownership	Updated whenever a task changes status
GitHub	Code, issues, pull requests and code review	Every working session

Channel / tool	Used for	Frequency
Notion / OneDrive	Shared documentation, meeting notes and report drafts	Updated after each meeting

Response expectations: messages are answered within 12 hours on weekdays and 24 hours on weekends. Pull requests are reviewed within 24 hours. If a member cannot attend a meeting or will miss a task deadline, they tell the other member at least 24 hours in advance and agree a new date.

On-track behaviours

- Finishing assigned tasks by the agreed date and moving the Trello card when the status changes.
- Committing small, frequent changes with clear semantic commit messages (feat:, fix:, docs:).
- Raising blockers early in the group chat instead of waiting until the deadline.
- Reviewing each other's work honestly and constructively before it is merged or submitted.
- Attending meetings on time and coming prepared with progress to show.

Off-track behaviours

- Missing deadlines without warning, or not responding to messages for more than 48 hours.
- Pushing untested code directly to the main branch.
- Working on tasks that are not on the board, or changing shared decisions without discussion.
- Leaving most of the work to the last day before a deadline.
- Submitting work generated by GenAI without understanding, checking or declaring it.

Resolving tensions

1. **Talk early:** the issue is raised privately and respectfully within 48 hours, focusing on the task, not the person.
2. **Agree an action:** both members agree on a concrete fix (e.g. redistribute a task, new internal deadline) and record it in the meeting notes.
3. **Use evidence:** Trello history and GitHub commits are used to keep the discussion factual.
4. **Escalate if needed:** if the problem continues after two attempts, we contact the tutor for mediation before the next deadline.

Team Expectations

#	Expectation
1	Both members contribute a balanced share of the work (about 50% each) across code, documentation and presentation.
2	All code goes through a pull request and is reviewed by the other member before merging into main.
3	Every deliverable is complete, reviewed by both members and submitted at least a few hours before the deadline.

2. System Blueprint Diagram

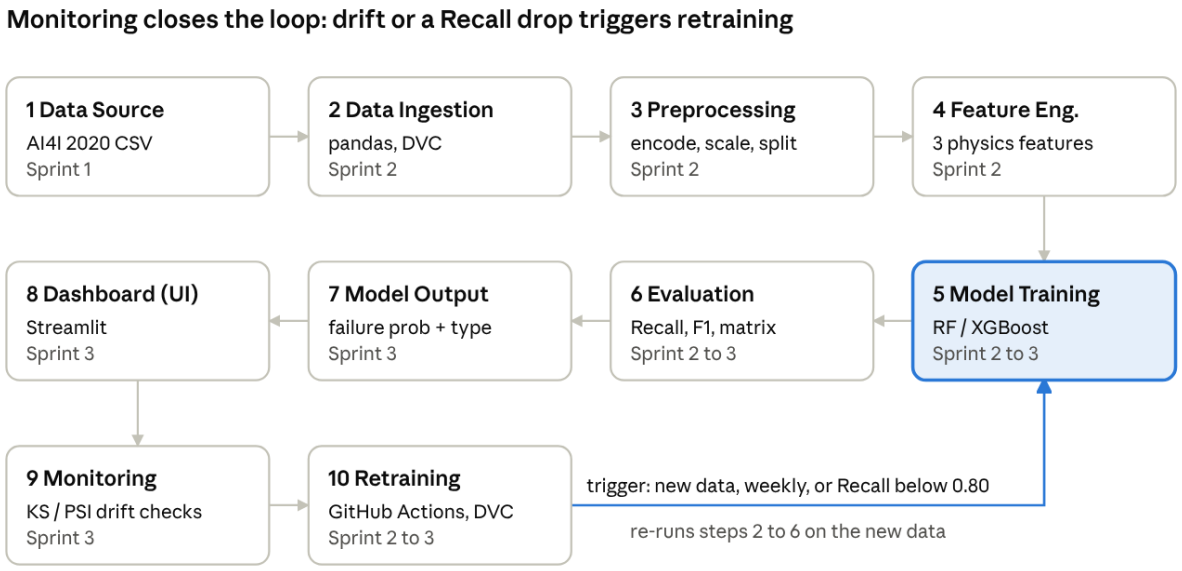


Figure 1. System blueprint of the predictive maintenance system: components, data flow, ML integration and the sprint in which each stage is built.

Diagram explanation

The blueprint (Figure 1) is read from the top left. Steps 1–4 prepare the data, steps 5–8 train the model and deliver its predictions, and steps 9–10 monitor the deployed model and retrain it. The highlighted box is the ML model component.

#	Component	What it does	Tools	Sprint
1	Data source	AI4I 2020 CSV (10,000 rows). Later, simulated new batches of sensor data are added to test retraining and drift.	Kaggle / UCI	1
2	Data ingestion	Loads raw CSV files, validates schema, ranges and missing values, and versions every dataset.	Python, pandas, DVC	2
3	Preprocessing	Drops UDI/Product ID and the five failure-mode columns (to avoid data leakage), one-hot encodes Type, scales numeric features (robust/log option for the right-skewed rotational speed), stratified 70/15/15 split.	pandas, scikit-learn	2
4	Feature engineering	Adds physics-based features: temp_diff = process − air temperature; power = torque × rotational speed × 2π/60; strain = tool wear × torque.	pandas	2
5	Model training	Random Forest and XGBoost classifiers with class weights or SMOTE (training set only) to handle the 3.39% failure rate.	scikit-learn, XGBoost	2–3
6	Model evaluation	Recall, precision, F1, PR-AUC and confusion matrix on the validation and test sets; results saved as metrics.txt and plots.	scikit-learn, matplotlib	2–3
7	Model output	Failure risk score, a high-risk flag and the predicted failure type(s) where available.	Python	3
8	Dashboard (UI)	Users enter or upload sensor readings and see the predicted risk and failure type(s).	Streamlit	3
9	Monitoring	Logs predictions, tracks Recall/F1 when labels arrive, and runs data drift checks on each new batch.	Python, Evidently (optional)	3
10	Retraining	Re-runs steps 2–6 when new data is pushed, weekly, on manual trigger, or when Recall drops below 0.80.	GitHub Actions, DVC	2–3

2.1 Key EDA findings

We ran an exploratory data analysis (EDA) on the full dataset (notebooks/Task1_EDA_AI4I.ipynb in the group repository). The dataset has 10,000 rows, 14 columns, no missing values and no duplicate rows. The findings below shaped the design of the preprocessing, feature engineering, metrics and monitoring stages.

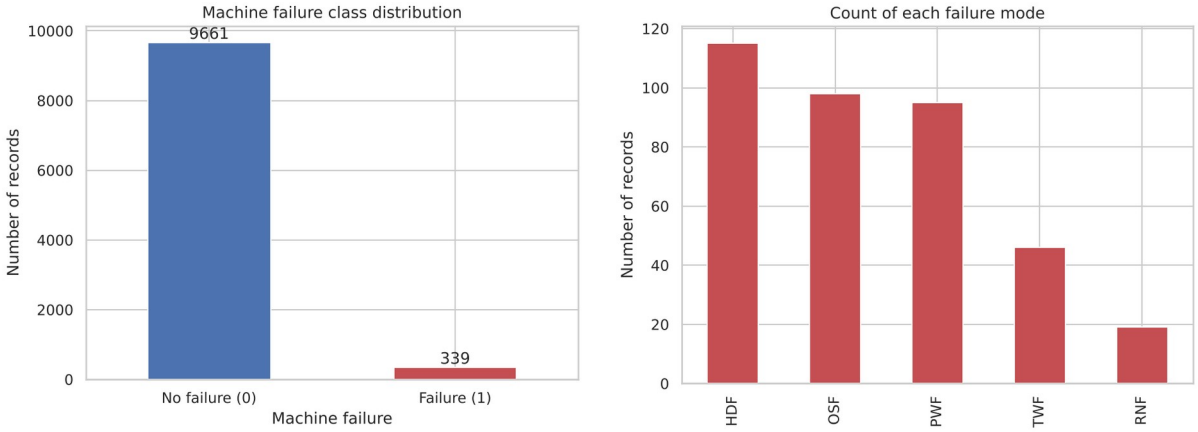


Figure 2. Left: only 339 of 10,000 records (3.39%) are failures. Right: HDF is the most common failure mode and RNF the rarest.

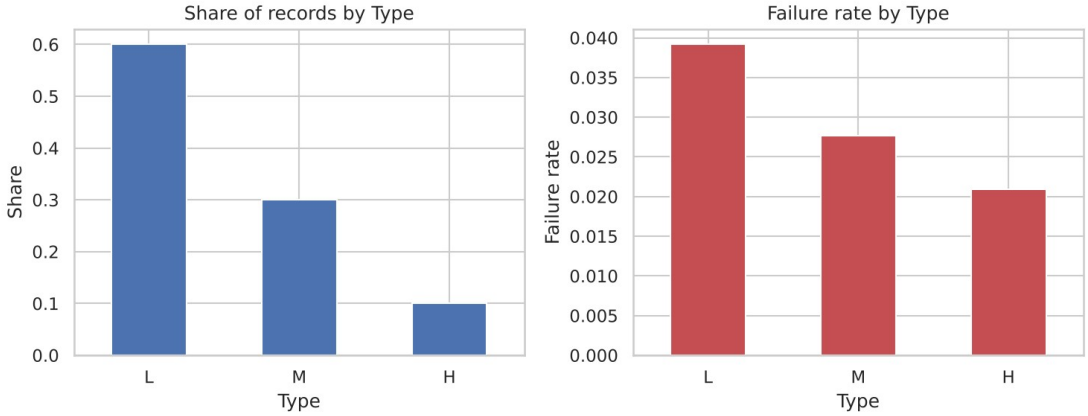


Figure 3. Type L makes up 60% of the data and has the highest failure rate (3.9%), compared with M (2.8%) and H (2.1%).

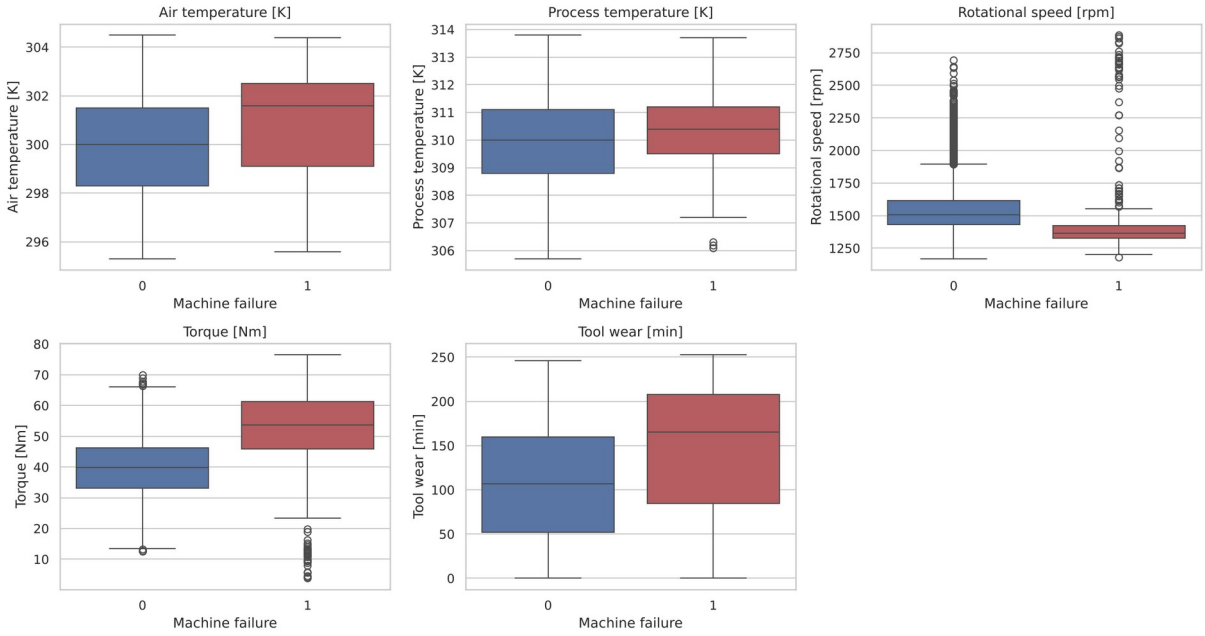


Figure 4. Failed machines have higher torque (mean 50.2 vs 39.6 Nm) and higher tool wear (143.8 vs 106.7 min).

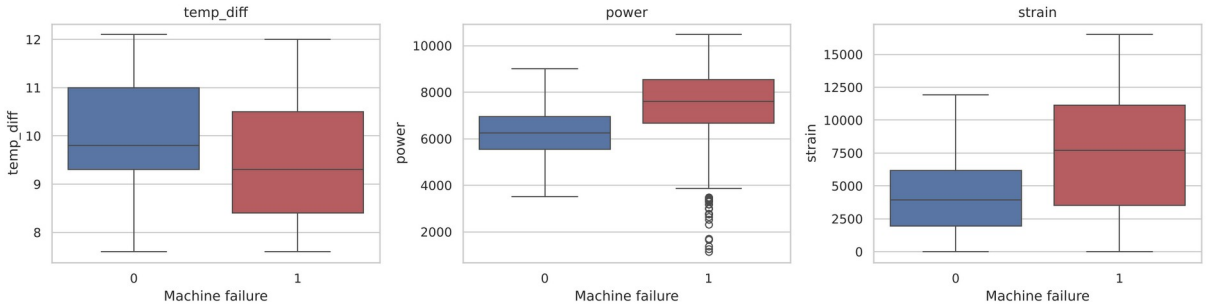


Figure 5. Engineered features: strain and power are clearly higher for failed machines; temp_diff is lower.

- **Class imbalance:** only 3.39% of records are failures, so a model that always predicts "no failure" would reach about 96.6% accuracy while being useless.
- **Label consistency:** 9 failures have no failure mode flagged, 18 records have RNF = 1 but Machine failure = 0, and 24 records have more than one failure mode.
- **Feature scales and skew:** the five numeric features use very different units; rotational speed is right-skewed (skewness ≈ 1.99, up to 2,886 rpm).
- **Correlation:** air and process temperature are strongly correlated (0.88), and rotational speed and torque are strongly negatively correlated (−0.88). No single feature is strongly correlated with failure (highest: torque, 0.19).
- **Engineered features:** strain (correlation 0.19) and power (0.18) are as informative as the best raw feature, and each lines up with its failure mode (mean strain 12,067 when OSF = 1 vs 4,238 otherwise).

2.2 How EDA insights informed the design

EDA insight (with numbers)	Design decision
Only 3.39% of records are failures (339 of 10,000).	Recall and F1 as the main metrics; stratified split; class weights or SMOTE on the training set only.
UDI and Product ID are unique identifiers with no predictive meaning.	Drop them in preprocessing.
Failure-mode columns define the target; labels have small inconsistencies (9 failures with no mode, 18 RNF records with no failure).	Exclude TWF/HDF/PWF/OSF/RNF from model inputs to avoid leakage; use them only as labels for the secondary failure-type model, treated as a multilabel problem because modes can overlap.
Failure rate differs by Type: L 3.9%, M 2.8%, H 2.1%.	Keep Type as a feature (one-hot encoded) and keep its proportions in every split.
Features have very different scales; rotational speed is right-skewed (skewness ≈ 2).	Scale numeric features; consider a robust scaler or log transform for rotational speed; monitor it closely for drift.
No single feature is strongly correlated with failure (max 0.19), but strain, power and temp_diff line up with OSF, PWF and HDF.	Add the engineered features temp_diff, power and strain; use tree-based models that capture feature interactions.

3. MLOps & DevOps Integration (High-Level Design)

Our MLOps design keeps the model accurate after it is first built: new data is versioned, the model is retrained automatically when needed, and every model version is traceable to the code and data that produced it.

3.1 Data & Training Strategy

- **Data split:** 70% training, 15% validation, 15% testing, stratified on “Machine failure” so every split keeps the 3.39% failure rate (about 237 failures in training and 51 each in validation and test). A fixed random seed makes the split reproducible.
- **Use of each split:** the training set fits the model; the validation set is used for hyperparameter tuning (GridSearchCV with stratified 5-fold cross-validation) and threshold selection; the test set is used once to report final performance.
- **Leakage control:** scaling and SMOTE are fitted on the training set only, inside a scikit-learn Pipeline. The five failure-mode columns are removed before training.
- **Class imbalance:** we will compare class weights and SMOTE on the validation set and keep the option with the higher Recall at an acceptable precision. The decision threshold is also tuned on the validation set, aiming for Recall ≥ 0.80 on the held-out test set (a target to be evaluated, not assumed).
- **New data for retraining:** yes. New batches of sensor data (simulated by sampling and perturbing the original data) are added to the data/ folder and versioned with DVC. Each retraining run uses all versioned data, and the test set is refreshed with a stratified sample of the newest batch so that evaluation reflects current conditions.

3.2 Retraining Strategy

Trigger type	Condition	Frequency	Mechanism
Data-based	A new validated, labelled data batch is pushed to the repository (data/ or its .dvc file changes)	On every data push	GitHub Actions workflow on push to main (path filter on data/)
Time-based	Scheduled retraining to absorb gradual change	Weekly (every Monday 02:00)	GitHub Actions schedule (cron)
Performance-based	Recall on newly labelled data falls below 0.80, or F1 below 0.70	Checked after each new labelled batch	Monitoring script raises a flag that starts the workflow
Manual	Team decides to retrain (e.g. after a feature change)	As needed	GitHub Actions workflow_dispatch button

Model promotion rule: a retrained model replaces the current model only if its test Recall and F1 are equal to or better than the current model’s. Otherwise the current model stays in use and the run is logged for review.

3.3 Monitoring & Drift Detection

- **Model performance monitoring:** every prediction is logged with its input features, probability and timestamp. When true labels become available, Recall, precision, F1 and PR-AUC are recomputed and compared with the baseline stored in metrics.txt.

- **Data drift (input distribution changes):** each new batch is compared with the training data. Numeric features use the Kolmogorov–Smirnov test (drift if $p < 0.05$) and the Population Stability Index (drift if PSI > 0.2); the Type feature uses a chi-square test. Torque and tool wear (the features that differ most between failed and healthy machines) and the right-skewed rotational speed are watched most closely.
- **Concept drift (relationship between inputs and failure changes):** detected when performance drops on newly labelled data while the input distribution looks similar, e.g. Recall falling below 0.80 over two consecutive batches.
- **Response:** drift alerts are written to a monitoring report and trigger a team review. Retraining only runs when suitable labelled data is available; a performance drop on labelled data starts the performance-based retraining trigger.
- **Demonstration:** because AI4I is synthetic, we will simulate drift for the Task 3 demo, for example by shifting air and process temperature by a few kelvin or increasing tool wear, and show that the checks detect it.

3.4 DevOps Practices

Version control strategy

- We will use **GitHub** for version control, with one shared group repository.
- **Branches:** main (stable, always runnable), dev (integration) and short-lived feature branches (feature/<task>), e.g. feature/feature-engineering.
- **Pull requests:** all changes reach dev and main through pull requests reviewed by the other member; direct pushes to main are not allowed.
- **Commits:** semantic commit messages (feat:, fix:, docs:, test:, refactor:, chore:) that reference the Trello card.
- **Repository layout:** data/ (DVC-tracked), src/ (pipeline code), notebooks/ (EDA), models/, tests/, docs/, .github/workflows/.

CI/CD pipeline concept

The CI/CD pipeline will be implemented with **GitHub Actions** in Task 2 and extended in Task 3. Conceptually, it runs these stages:

1. **Trigger:** push or pull request to main, a weekly schedule, a new data push, or a manual workflow_dispatch.
2. **Set-up:** check out the code on ubuntu-latest, install Python and requirements.txt, pull the versioned data with DVC.
3. **Test (CI):** run linting and unit tests for preprocessing and feature functions.
4. **Train and evaluate:** run model.py, then save metrics.txt and result plots (e.g. model_results.png).
5. **Quality gate:** compare the new metrics with the targets (Recall ≥ 0.80, F1 ≥ 0.70) and the current model; a run that does not pass is marked as not promotable.
6. **Publish (CD):** upload the model and metrics as workflow artifacts, tag the new model version, and update the dashboard's model file.

Code and model versioning

- **Code:** versioned with Git; each release is tagged with semantic versions (v1.0.0, v1.1.0, …).

- **Data:** versioned with DVC; the .dvc pointer files are committed to Git and the data itself is stored in a DVC remote (e.g. DagsHub).
- **Models:** each trained model file is tracked with DVC and tagged (e.g. model-v1.2) together with its metrics.txt, the Git commit hash and the data version used. This lets us reproduce or roll back any model.

4. Project Management Setup (Agile Tools)

4.1 Trello board

- **Board link:** https://trello.com/b/VTa0ps5c/cos40007-hcmc6
- **Lists:** Backlog → To Do (current sprint) → In Progress → Review → Done.
- **Labels:** priority (High / Medium / Low) and sprint (S1–S4).
- **Cards:** one card per task, with a member assigned, a due date and a link to its user story.

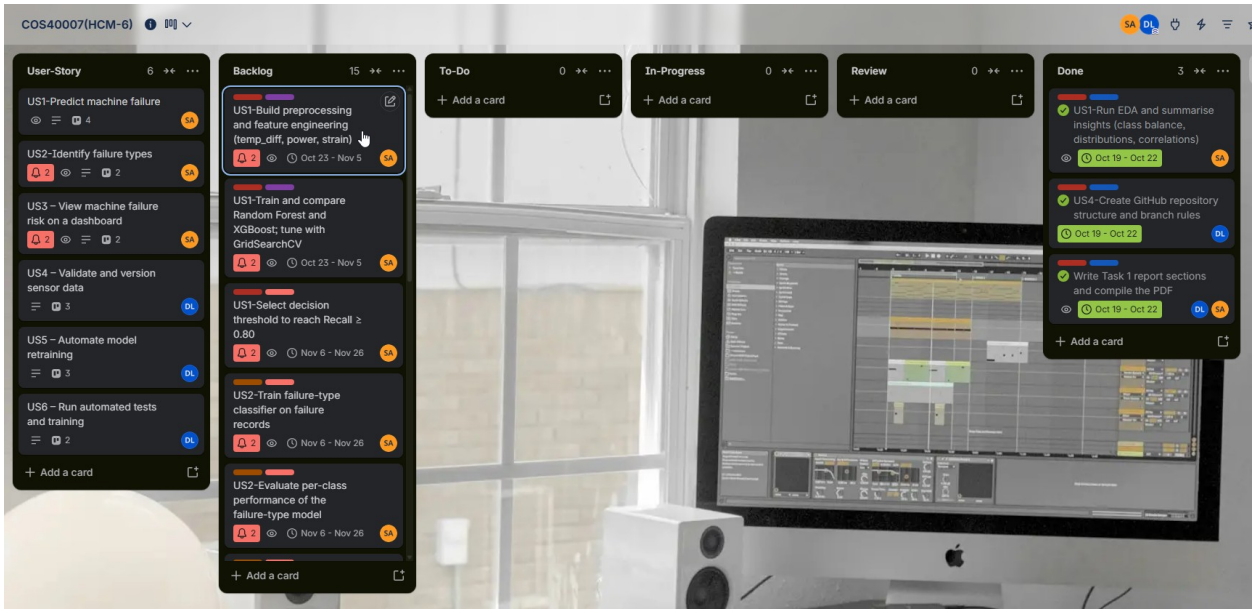


Figure 6. Trello board (Board view): user stories, backlog and completed Sprint 1 tasks with priority/sprint labels, members and due dates.

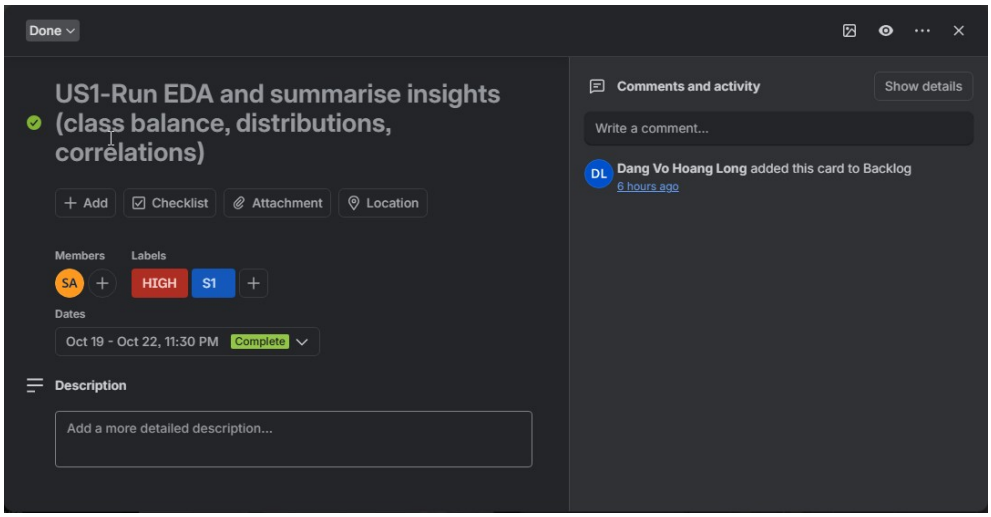


Figure 7. Example task card showing its assigned member, labels (HIGH, S1), dates and completion status.

4.2 User stories

ID	User story	Priority	Sprint	Owner	Acceptance criteria
US1	As a **maintenance engineer**, I want to be warned before a machine is likely to fail so that I can schedule repairs and avoid unplanned downtime.	High	S2–S3	Huy	Model reaches Recall ≥ 0.80 on the test set; high-risk machines are flagged.

ID	User story	Priority	Sprint	Owner	Acceptance criteria
US2	As a **maintenance engineer**, I want to know the likely failure type(s) so that I can prepare the right parts and fix.	Medium	S3	Huy	Dashboard shows the predicted failure type(s) for high-risk inputs.
US3	As an **operations manager**, I want a simple dashboard showing each machine's failure risk so that I can prioritise maintenance work.	Medium	S3	Huy	Streamlit dashboard accepts inputs/CSV and shows risk levels.
US4	As a **data engineer**, I want new sensor data to be validated and versioned automatically so that every model is trained on clean, traceable data.	High	S2–S3	Long	Invalid batches are rejected; every dataset version is tracked by DVC.
US5	As an **ML team member**, I want the model to retrain automatically when new data arrives or performance drops so that predictions stay accurate over time.	High	S3	Long	Workflow retrains on data push, schedule and manual trigger; promotion rule applied.
US6	As a **developer**, I want every push to main to run tests and training automatically so that broken code never reaches the stable branch.	High	S2	Long	GitHub Actions runs on push/PR and uploads metrics.txt and plots.

4.3 Backlog & task breakdown

Story	Task	Owner	Priority	Sprint
US1	Run EDA and summarise insights (class balance, distributions, correlations)	Huy	High	S1
US1	Build preprocessing and feature engineering (temp_diff, power, strain)	Huy	High	S2
US1	Train and compare Random Forest and XGBoost; tune with GridSearchCV	Huy	High	S2
US1	Select decision threshold to reach Recall ≥ 0.80	Huy	High	S3
US2	Train failure-type classifier on failure records	Huy	Medium	S3
US2	Evaluate per-class performance of the failure-type model	Huy	Medium	S3
US3	Design dashboard layout (input form, CSV upload, risk display)	Huy	Medium	S3
US3	Build Streamlit dashboard and connect it to the saved model	Huy	Medium	S3
US4	Create GitHub repository structure and branch rules	Long	High	S1
US4	Write ingestion and schema/range validation script	Long	High	S2
US4	Set up DVC with a remote (DagsHub) and track data/	Long	High	S3
US5	Write drift detection script (KS-test, PSI, chi-square)	Long	High	S3
US5	Add data-push, schedule and manual triggers to the workflow	Long	High	S3
US5	Implement model promotion rule and model version tags	Long	Medium	S3
US6	Write requirements.txt and unit tests for preprocessing	Long	High	S2
US6	Create .github/workflows/train.yml to train and upload metrics/plots	Long	High	S2

Story	Task	Owner	Priority	Sprint
All	Write Task 1 report sections and compile the PDF	Huy & Long	High	S1
All	Final report, slides and live demo rehearsal	Huy & Long	High	S4

4.4 Project documentation

Documentation link: https://app.notion.com/p/COS40007-HCM-6-40152f6e95ab8061ae41c3954da15b2a

The shared Notion space contains:

- **Project description:** problem, dataset, ML tasks, users and success criteria (as in Section 1).
- **System overview:** the blueprint diagram and component table (Section 2).
- **Key design decisions:** summarised below, with the reason for each.
- **Meeting notes and sprint reviews:** updated after every meeting.

Key design decision	Reason
Recall and F1 as the main metrics	A missed failure costs much more than a false alarm, and accuracy is misleading at a 3.39% failure rate.
Exclude failure-mode columns from model inputs	They define the target and would cause data leakage.
Tree-based models (Random Forest, XGBoost)	They handle feature interactions and different scales well, and are explainable through feature importance.
GitHub Actions for CI/CD	Required by the unit, free for our repository and integrated with GitHub pull requests.
DVC for data and model versioning	Keeps large files out of Git while linking every model to the exact data version.
Streamlit for the dashboard	Fast to build in Python and easy to demonstrate live.

5. Contribution Statement

Both members agree that the work for Task 1 was shared in a balanced way.

Member	Contribution to Task 1	Share
Phan Quang Huy	• Cover page, Acknowledgement of Country and GenAI declaration • EDA and EDA-informed design decisions • System blueprint diagram and explanation • Data & training strategy, retraining strategy, monitoring & drift detection • User stories US1–US3 and their tasks • Compiling, formatting and submitting the report	50%
Đặng Võ Hoàng Long	• Group agreement (communication plan, behaviours, resolving tensions, team expectations) • DevOps practices: version control, CI/CD concept, code and model versioning • GitHub repository set-up • Trello board set-up and screenshots • User stories US4–US6 and their tasks • Notion project documentation and final review of the report	50%
Both	Project definition, roles, sprint plan, contribution statement and final review.	–

Agreed by: Phan Quang Huy and Đặng Võ Hoàng Long, 22 October 2026.

6. Group GitHub Repository

- **Repository link:** https://github.com/long-blip/COS40007-HCM6
- This report (PDF) and the blueprint diagram are stored in the docs/ folder; the EDA notebook and its figures are in notebooks/.

7. GenAI Declaration

We used Claude (Anthropic) as a GenAI assistant while preparing this task. It was used to help understand the task brief and rubric, suggest a task split and order of work, draft and structure report sections, propose user stories, produce a first version of the system blueprint diagram, and draft the written observations in the EDA notebook from our own notebook outputs. All content was reviewed, edited and checked by both group members, and the design decisions reflect our own agreement. We take full responsibility for the submitted work.

References

Matzka, S. (2020a). AI4I 2020 Predictive Maintenance Dataset [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5HS5C

Matzka, S. (2020b). Explainable artificial intelligence for predictive maintenance applications. 2020 Third International Conference on Artificial Intelligence for Industries (AI4I), 69–74.

Kaggle. (n.d.). Predictive Maintenance Dataset (AI4I 2020). https://www.kaggle.com/datasets/stephanmatzka/predictive-maintenance-dataset-ai4i-2020